Devups

Tome I — Le socle et le démarrage

Documentation du framework

Première édition — novembre 2026

Établie à partir du code réellement exécuté dans le projet **3agReader** (socle Devups 4.3).
Chaque affirmation porte sa source : *fichier:ligne*.

Sommaire

Avant-propos 4
- Ce que ce document est 4
- Comment il a été produit 4
- Comment le lire 4
- Ce que ce document ne couvre pas 5

Chapitre 1 — Qu'est-ce que Devups 6
- 1.1 En une phrase 6
- 1.2 Les quatre idées qui le caractérisent 6
- 1.3 Les deux niveaux : composant et module 6
- 1.4 Prérequis 7

Chapitre 2 — Anatomie d'un projet 9
- 2.1 L'arborescence racine 9
- 2.2 La frontière qui compte : dclass/ contre src/ 9
- 2.3 L'intérieur de dclass/ 10
- 2.4 L'intérieur d'un module 10
- 2.5 Les deux fichiers de configuration générés 11

Chapitre 3 — Installation 13
- 3.1 La séquence officielle 13
- 3.2 Ce que install fait réellement, et où il échoue 13
- 3.3 Les dossiers à créer 13
- 3.4 Le serveur de développement 14
- 3.5 Vérifier que l'installation tient 14

Chapitre 4 — Configuration 15
- 4.1 Les fichiers 15
- 4.2 Les constantes, par famille 15
- 4.3 L'ordre de chargement 17
- 4.4 Le mode production 18

Chapitre 5 — Le cycle de vie d'une requête 19
- 5.1 Les deux points d'entrée 19
- 5.2 Les quatre portes 19
- 5.3 Porte 1 — App::run(), les pages publiques 19
- 5.4 Porte 2 — WebService::runServe(), l'API publique 20
- 5.5 Porte 3 — Admin::manage(), les pages du back-office 21
- 5.6 Porte 4 — AdminService::manageServe(), l'API du back-office 22
- 5.7 mapFunction() — le dernier kilomètre 22
- 5.8 Le cache de routes 23
- 5.9 Résumé en un tableau 23

Chapitre 6 — La ligne de commande 24
6.1 Le binaire 24
6.2 Les commandes de cycle de vie 24
6.3 Les commandes de schéma 24
6.4 Les commandes du datacore 25
6.5 Les commandes de génération 25
6.6 Le garde de régénération, et l'option --force 26
6.7 Ce qu'il faut savoir avant de régénérer 27

Chapitre 7 — Le générateur et le scan du disque 28
7.1 Les trois niveaux de datacore 28
7.2 Le scan, mesuré 28
7.3 Les trois conséquences à retenir 28
7.4 Créer un module à la main 29
7.5 Ce que dvups_:update écrit vraiment 29
7.6 Les trois choses que le datacore ne sait pas exprimer 30

Chapitre 8 — Premier module, de bout en bout 31
8.1 Le datacore du composant 31
8.2 Le datacore du module 31
8.3 Le datacore de l'entité 31
8.4 Générer 32
8.5 Les quatre gestes manuels 32
8.6 Migrer la base 33
8.7 Vérifier 33
8.8 Si quelque chose ne marche pas 33

Avant-propos

Ce que ce document est

Devups est un framework PHP écrit par Aurelien Atemkeng (Spacekola). Il n'a pas de documentation publique à jour. Ce document est la première tentative d'en écrire une complète, à partir du code réellement exécuté dans un projet en production : **3agReader** (3ag édition), une plateforme de bandes dessinées et de mangas dont le dépôt `backend/` porte le socle Devups en version 4.3.

Il est écrit pour deux lecteurs :

- **un développeur qui reprend un projet Devups** et doit comprendre où les choses se passent avant de pouvoir y toucher ;
- **l'auteur du framework lui-même**, pour qui ce document sert de miroir : il nomme ce que le code fait réellement, y compris là où ce n'est pas ce que le code avait l'air de vouloir faire.

Comment il a été produit

Chaque affirmation de ces quatre tomes provient d'une lecture du code source, pas d'une documentation antérieure ni d'une supposition. Quand une affirmation porte une référence de la forme `QueryBuilder.php:1582`, c'est le fichier et la ligne où le comportement décrit est écrit.

Trois sources antérieures ont été intégrées, et sont nommées quand elles le sont :

- `backend/DEVUPS_GUIDE.md` — un mémo interne écrit en travaillant sur **un autre projet Devups** (BGFIBank Queue). Il contient des mesures faites à l'exécution sur ce projet-là. Quand ce document s'en sert, il le dit, et il signale les endroits où le code de 3agReader a **évolué depuis** et contredit le mémo.
- `backend/DEVUPS_TCHUTTE_ORM_DOC.md` — une référence de la couche d'accès aux données, également mesurée à l'exécution sur BGFIBank Queue.
- Les trois protocoles `PROTOCOLE_moduletree.md`, `PROTOCOLE_dvtranslate.md`, `PROTOCOLE_status.md` — écrits sur 3agReader, intégrés au Tome IV.

Comment le lire

Le découpage suit celui de la documentation Laravel, dont ce document reprend volontairement la forme : **un chapitre explique un concept, montre un exemple complet tiré du vrai code, puis donne le tableau de référence exhaustif.**

Tome	Contenu
I — Le socle et le démarrage	ce qu'est Devups, l'anatomie d'un projet, l'installation, la configuration, le cycle de vie d'une requête, la ligne de commande, le générateur
II — Les données	le datacore, les entités, le `QueryBuilder`, l'écriture, les transactions, la traduction

Tome	Contenu
III — L'interface	le routage, les contrôleurs, les deux styles d'API, les listes, les formulaires, les vues, la sécurité
IV — Les modules fournis	les composants du socle, `ModuleTree`, `ModuleData`, le protocole de cache, les dettes connues

Ce que ce document ne couvre pas

Nommer les trous est plus utile que de les combler à l'aveugle. Les zones suivantes ne sont pas documentées ici, et c'est délibéré :

- **Le générateur Android/Java** (`dclass/generator/android/`) — présent, aucun usage mesuré dans 3agReader.
- **`Dfile.php` et `UploadFile.php`** (31 ko et 29 ko) — la gestion des fichiers et des images est survolée au Tome III, pas traitée en référence.
- **La couche JavaScript du back-office** (`dclass/devupsjs/`) — décrite par son rôle au Tome III, pas en référence d'API.
- **`DBAL.php`** (60 ko) — son contrat public est documenté, son intérieur non.
- **Le déploiement en production** — les commandes `build` / `deploy` existent, leur usage réel sur un serveur n'a pas été observé.

Ce document est destiné à être **itéré**. Chaque projet Devups suivant devrait y ajouter ce qu'il mesure, et corriger ce qu'il trouve faux.

Chapitre 1 — Qu'est-ce que Devups

1.1 En une phrase

Devups est un framework PHP dans lequel **l'arborescence des dossiers est le modèle de données**, et où un générateur de code produit, à partir de fichiers JSON de description, l'entité, le formulaire, la liste et les contrôleurs d'un écran d'administration complet.

1.2 Les quatre idées qui le caractérisent

1. Le datacore. Un dossier n'existe pour le framework que s'il contient un fichier `*Core.json` qui le déclare. Le disque est scanné à chaque démarrage, et c'est ce scan — pas une table de configuration, pas un fichier de routes — qui établit la liste des composants, des modules et des entités du projet. Un module sans `<module>Core.json` est invisible : silencieusement, sans erreur.

2. Le contrat déclaratif. Une liste de back-office ne s'écrit pas en HTML : elle se déclare en PHP sous forme de tableau (`$datatablemodel`), et un moteur unique (`Datatable.php`) rend toutes les listes du projet. Un formulaire se déclare de la même façon (`$this->fields`), et un moteur unique (`FormFactory.php`) rend tous les formulaires. Changer le moteur change toutes les listes et tous les formulaires d'un coup — ce qui est à la fois la grande force et le grand risque du framework.

3. Deux styles d'API qui cohabitent. Un style générique (`POST /api/create/comicbook`) sert n'importe quelle entité enregistrée, sans écrire une ligne de contrôleur. Un style REST dédié (`POST /api/comicbook`) passe par un `FrontController` par entité, piloté par annotations. Les deux sont vivants dans le même projet, sur les mêmes entités, et ne se comportent pas pareil. Le Tome III les sépare proprement.

4. Doctrine n'est là que pour les métadonnées. `doctrine/orm` est bien dans les dépendances, et c'est lui qui lit les annotations `@Column` / `@ManyToOne` et qui applique les migrations. Mais **aucune requête métier ne passe par Doctrine** : toute lecture et toute écriture passent par la couche maison `DBAL` / `QueryBuilder`, dite « Tchutte » (du surnom de l'auteur du socle, pas d'un produit externe).

1.3 Les deux niveaux : composant et module

Un projet Devups est découpé en **composants** (`component`), eux-mêmes découpés en **modules**.

- Un **composant** est un métier. Dans 3agReader : `devups` (le noyau d'administration), `devupscms` (le contenu), `devupsfront` (les utilisateurs), `shopping` (le commerce), `edition3ag` (le métier applicatif — les bandes dessinées), `spacekola`.
- Un **module** est une fonctionnalité à l'intérieur d'un composant : `ModuleComicbook`, `ModuleOrder`, `ModuleWallet`, `ModuleTree`.

Le composant principal porte généralement le nom du projet. Les composants préfixés `devups*` sont **le socle réutilisable** : ils sont censés arriver identiques dans tout nouveau projet.

> **Convention, pas contrainte technique.** Rien n'empêche de ranger un module dans un composant auquel il n'appartient pas logiquement. Le mémo de BGFIBank en fait une règle d'hygiène, pas une règle du framework : « ne jamais éparpiller un module dans un composant auquel il n'appartient pas — même si ça marcherait techniquement ».

1.4 Prérequis

Élément	Valeur
PHP	7.x minimum (le `README.md` du socle annonce « 5.6, 7.^ » ; le code de 3agReader utilise des fonctions de PHP 8, par ex. `str_contains()` dans `Router.php:127`)
Base de données	MySQL / MariaDB
Composer	requis
Connaissances	PHP objet, UML (annoncé par le `README.md`)

Dépendances principales (`composer.json`) :

Paquet	Rôle
`doctrine/orm` ^2.1.3	métadonnées d'annotation et migrations **uniquement**
`philo/laravel-blade` 3.*	moteur de vues Blade
`firebase/php-jwt` ^6.9	jetons d'authentification de l'API
`mpdf/mpdf` ^8.0	génération de PDF
`phpmailer/phpmailer` ^6.6	envoi de courriels
`guzzlehttp/guzzle` ^7.5	appels HTTP sortants
`maslosoft/addendum`	lecture des annotations de route (`ReflectionAnnotatedClass`)
`ifsnop/mysqldump-php` ^2.9	sauvegardes de base
`zircote/swagger-php` ^4.11	annotations OpenAPI
`google/apiclient` ^2.18	authentification Google
`barcode-bakery/barcode-1d` ^7.0	codes-barres

L'autochargement est un **classmap** sur `dclass/` et `src/`, plus un `psr-0` vide sur `src/`. Conséquence pratique, et piège réel : Composer résout une classe **par son nom**, pas par le composant qui la déclare. Deux projets Devups sur le même poste qui partagent un dossier `vendor/` hérité peuvent voir une entité d'un projet intercepter celle de l'autre. Le symptôme est un `Class 'X' does not exist` qui pointe vers un chemin d'un autre projet, alors que la classe est bien sur le disque au bon endroit. Le correctif est `composer dump-autoload` depuis `backend/`.

Chapitre 2 — Anatomie d'un projet

2.1 L'arborescence racine

```
backend/
├── admin/            le back-office (point d'entrée, vues, assets)
├── bin/              binaire Doctrine (bin/doctrine)
├── cache/            vues Blade compilées, routes, arbres compilés
├── config/           configuration et injection de dépendances
├── database/         dumps, migrations, logs, transactions
├── dclass/           LE FRAMEWORK — ne se modifie pas par projet
├── documentation/    un guide HTML servi par l'application
├── route/            les quatre portes d'entrée du routage
├── src/              LE PROJET — composants, modules, entités
├── tests/
├── uploads/          fichiers déposés par les utilisateurs
├── vendor/           dépendances Composer
├── web/              le site public (vues, assets)
├── composer.json
├── cron.php          tâches planifiées
├── devups            le binaire de ligne de commande
├── header.php        amorçage de l'application publique
├── index.php         point d'entrée public
└── lang.php          fonctions de traduction
```

2.2 La frontière qui compte : `dclass/` contre `src/`

C'est la distinction la plus importante du framework, et la plus facile à enfreindre.

`dclass/` est le framework. Il arrive identique dans tout projet Devups. Y toucher répare un défaut pour un projet et le réintroduit pour tous les autres — ou, pire, corrige un comportement sur lequel un autre projet s'appuie.

`src/` est le projet. C'est là que vit tout ce qui est propre à l'application.

> **Règle de travail.** Quand une entité a besoin d'un accesseur que sa classe parente de `dclass/extends/` ne fournit pas, l'accesseur s'ajoute **sur l'enfant, dans `src/`**, jamais sur le parent. Cette règle a été appliquée sur 3agReader pour `User::getUsername()` et `Orders::getTotalamount()`, dont les colonnes sont déclarées dans `dclass/extends/UserCore.php` et `OrderCore.php`.

2.3 L'intérieur de `dclass/`

```
dclass/
├── devups/
│   ├── Core.php              le scan du datacore
│   ├── Genesis.php           le rendu des vues (Blade) et des réponses JSON
│   ├── ScanDir.php
│   ├── Api.php
│   ├── Dfile.php             fichiers
│   ├── UploadFile.php        envois de fichiers
│   ├── System_prod.php
│   ├── Controller/
│   │   ├── Controller.php        contrôleur d'administration (CRUD générique)
│   │   ├── FrontController.php   contrôleur d'API (CRUD générique)
│   │   ├── CrudTrait.php
│   │   └── ModuleController.php
│   ├── Datatable/
│   │   ├── Datatable.php         LE moteur de listes (55 ko)
│   │   ├── Lazyloading.php       pagination et filtres dynamiques
│   │   ├── Dbutton.php, TableRow.php, TableData.php
│   ├── Form/
│   │   ├── FormManager.php       la classe de base d'un formulaire + le vocabulaire
│   │   ├── FormFactory.php       LE moteur de rendu des champs (32 ko)
│   │   └── Form.php              l'API « à la main », hors contrat déclaratif
│   ├── Http/
│   │   ├── Router.php            les quatre dispatchers (28 ko)
│   │   ├── Request.php           la requête + le dispatch REST dédié
│   │   ├── Route.php             l'appariement d'un motif d'URL
│   │   └── Response.php
│   ├── model/
│   │   ├── Model.php             la classe de base de toute entité
│   │   ├── ModelTrait.php        les points d'entrée statiques (42 ko)
│   │   ├── ModelSerializer.php   sérialisation JSON + insert/update/delete
│   │   ├── EntityCollection.php, EntityCore.php, DateTimeTrait.php
│   │   └── DvtranslateInspector.php   outil de diagnostic (ajouté par 3agReader)
│   └── Tchutte/
│       ├── QueryBuilder.php      la construction du SQL (64 ko)
│       ├── DBAL.php              l'exécution et l'hydratation (60 ko)
│       ├── DB_dumper.php, Column.php, Bugmanager.php
├── devupsjs/            le JavaScript du back-office
├── extends/             les classes de base d'entités réutilisables
├── generator/           le générateur de code
└── lib/
    ├── annotation/      GET, POST, PUT, DELETE, Api, Auth, MethodView, DDoc
    ├── addendum/        le lecteur d'annotations (vendu avec)
    └── Util.php, DMail.php, DPdf.php, HZip.php, SimpleXLSX.php
```

2.4 L'intérieur d'un module

C'est la forme que le générateur produit, et celle qu'un module écrit à la main doit respecter.

```
src/<composant>/<Module>/
├── <module minuscule>Core.json     LE FICHIER QUI FAIT EXISTER LE MODULE
├── Module<Nom>.php                 la classe du module (vues, métadonnées)
├── <composant>.module<nom>.php     les `require` des classes du module
├── services.php                    les routes de service du back-office
├── Core/
│   └── <Entité>Core.json           le datacore de chaque entité
├── Entity/
│   ├── <Entité>.php
│   └── <Entité>_lang.php           si l'entité est traduite
├── Controller/
│   ├── <Entité>Controller.php          back-office
│   └── <Entité>FrontController.php     API REST dédiée
├── Datatable/
│   └── <Entité>Table.php
├── Form/
│   └── <Entité>Form.php
└── Resource/
    ├── js/
    └── views/
        └── admin/
```

Exemple réel, `src/shopping/ModuleOrder/` :

```
ModuleOrder/
├── moduleorderCore.json
├── ModuleOrder.php
├── buyamsellam24.moduleorder.php
├── services.php
├── Core/            OrdersCore.json, Orders_itemCore.json, subscriptionCore.json
├── Entity/          Orders.php, Orders_item.php, Subscription.php
├── Controller/      OrdersController.php, OrdersFrontController.php,
│                    SubscriptionController.php
├── Datatable/       OrderTable.php, SubscriptionTable.php, Order_productTable.php
├── Form/            OrderForm.php, SubscriptionForm.php
└── Resource/        js/, views/admin/
```

> **À remarquer.** Le nom du fichier `<composant>.module<nom>.php` porte le nom du composant d'origine, pas celui du projet courant : `buyamsellam24.moduleorder.php` dans un projet nommé `shopping`. C'est un vestige du projet où ce module a été écrit la première fois ; le framework ne s'en soucie pas, le fichier est chargé par son chemin.

2.5 Les deux fichiers de configuration générés

`Core::updateDvupsTable()` (`Core.php:184`) écrit deux fichiers PHP à partir du scan du disque :

Fichier	Indexé par	Lu par
`config/dvups_configurations.php`	nom d'entité **et** nom pleinement qualifié	`Router::manage()` pour router un écran d'entité, `FormManager::__construct()` pour connaître le nom d'affichage
`config/module_configurations.php`	nom de module	`Router::manage()` pour router un écran de module, la barre latérale du back-office

Chaque entrée d'entité porte `path`, `module`, `name`, `label`, `namespace`, `namespace_name`, `component`. Chaque entrée de module porte `name`, `label`, `icon`, `entities`.

Ces deux fichiers sont **générés** — les éditer à la main ne sert à rien, la commande `php devups dvups_:update` les réécrit depuis le disque.

Chapitre 3 — Installation

3.1 La séquence officielle

D'après le `README.md` du socle :

```
cd <racine du projet>
composer install
php devups install
php devups dvups_:update
```

Avant cela, `config/constante.php` doit être édité (voir le chapitre 4).

3.2 Ce que `install` fait réellement, et où il échoue

`php devups install` fait trois choses en séquence :

1. crée la base de données si elle n'existe pas ;
2. lance `bin/doctrine orm:schema:create`, qui crée **toutes les tables, vides** — y compris `dvups_admin`, `dvups_role`, `dvups_lang` ;
3. exécute `dclass/generator/config_data.sql` en une seule requête PDO, pour semer le compte administrateur maître.

> **Piège vécu (mesuré sur BGFIBank Queue, non revérifié sur 3agReader).** L'étape 3 échoue silencieusement. `config_data.sql` n'est pas un script d'`INSERT` : c'est un dump complet qui **commence par ses propres `CREATE TABLE dvups_admin` / `dvups_lang` / `dvups_role`**. Sur une base où l'étape 2 vient de créer ces mêmes tables, la première instruction échoue (`SQLSTATE[42S01] Base table or view already exists`), l'exception est absorbée par un `try/catch`, et **aucune ligne `INSERT` du fichier ne s'exécute**. Résultat : le schéma existe, mais la table des administrateurs est vide et la connexion à `/admin/` est impossible. Le message de succès d'`install` (« Set the master admin... login: dv_admin password: admin ») s'affiche quand même : il est câblé après le `try/catch`.
>
> **Rattrapage constaté** : extraire à la main les trois `INSERT` de `config_data.sql` (`dvups_lang` ×2, `dvups_role` ×1, `dvups_admin` ×1, avec `login=dv_admin` et un mot de passe déjà haché) et les rejouer directement, en sautant les `CREATE TABLE` que Doctrine a déjà satisfaits. `php devups dvups_:admin` reproduit l'échec à l'identique : même fichier, même cause.

3.3 Les dossiers à créer

`php devups init:caches` crée l'arborescence que l'application attend :

```
cache/
├── views/          vues Blade compilées
├── trees/          arbres ModuleTree compilés (voir Tome IV)
└── local/
    ├── admin/
    └── front/
database/
├── dump/
├── log/
├── migration/
└── transaction/
```

3.4 Le serveur de développement

```
php devups serve
```

Lance le serveur PHP intégré. Sur 3agReader il écoute sur le port déclaré par `__env_port` (`:3334`), donc `http://127.0.0.1:3334/`.

3.5 Vérifier que l'installation tient

Vérification	Attendu
`php devups schema:update --dump-sql`	`Nothing to update - your database is already in sync`
`http://<hôte>/admin/login.php`	formulaire de connexion
connexion `dv_admin` / `admin`	redirection vers `complete-registration` (`firstconnexion=1`)
`GET /api/hello`	`{"success":true,"message":"hello devups you made your first apicall"}` (`route/Admin.php`, méthode `helloService()`)

Le critère « aucun diff en attente » sur `schema:update --dump-sql` n'a de valeur que s'il est **réellement vide**. Le chapitre 7 explique comment il peut se remplir de bruit permanent, et comment l'éviter.

Chapitre 4 — Configuration

4.1 Les fichiers

Fichier	Rôle	Suivi par git ?
`config/constante.php`	toutes les constantes du projet : base, URLs, secrets	**non** (ignoré)
`config/constante.prod.php`	gabarit de déploiement, copié en `constante.php` sur le serveur	oui
`config/dependanceInjection.php`	charge `constante.php`, Composer, puis **tout** `dclass/` dans l'ordre	oui
`config/Database.php`	la classe `Database` : ouvre la connexion PDO, crée la base si absente	oui
`config/bootstrap.php`	amorçage pour le binaire Doctrine	oui
`config/php_ini.php`	active l'affichage des erreurs (non requis par défaut)	oui
`config/dvups_configurations.php`	**généré** — voir §2.5	oui
`config/module_configurations.php`	**généré** — voir §2.5	oui

> **Attention, conséquence réelle.** `constante.php` étant ignoré par git, **une constante déclarée là n'existe pas sur un clone neuf**. C'est arrivé sur 3agReader : `KOLACOIN_RATE_FCFA` y vivait, et la première écriture d'une entité d'argent partait en « Undefined constant » sur tout nouveau poste. La constante a été déplacée dans `config/dependanceInjection.php`, qui est suivi par git et requis par **tous** les points d'entrée. C'est le bon endroit pour toute constante applicative qui doit survivre à un clone.
>
> `constante.prod.php` n'est **jamais** chargé par l'application : aucun `require` du dépôt ne le cible. C'est un gabarit, copié à la main au déploiement.

4.2 Les constantes, par famille

Les valeurs ci-dessous sont des **exemples de forme**. Les valeurs réelles d'un projet ne doivent jamais être recopiées dans un document : `constante.php` de 3agReader contient des identifiants d'hébergement, des mots de passe de messagerie, un secret JWT et des clés de services tiers en clair.

> **Observation de sécurité, à traiter.** Sur 3agReader, `config/constante.php` porte en tête un bloc de commentaires contenant des identifiants et mots de passe réels (hébergement, comptes de messagerie, magasin de clés Android). Le fichier est bien ignoré par git, ce qui limite la portée — mais un bloc de mots de passe en clair dans un fichier de configuration reste une sauvegarde qui part avec toute copie du dossier. Un gestionnaire de secrets, ou au minimum un fichier séparé hors du dossier du projet, serait préférable.

Identité du projet

Constante	Exemple	Rôle
`PROJECT_NAME`	`"MonProjet"`	nom affiché
`__v`	`'4.3'`	version du socle
`__project_id`	`'monprojet' . __env_port`	identifiant unique, sert de préfixe aux clés de session

Base de données

Constante	Exemple	Rôle
`dbname`	`'monprojet_bd'`	nom de la base
`dbuser` / `dbpassword` / `dbhost`	`'root'` / `''` / `'localhost'`	connexion
`dbdumper`	`false`	si `true`, un dump est pris à chaque requête (`dependanceInjection.php`)
`dbtransaction`	`true`	active le journal de transactions

URLs et environnement

Constante	Exemple	Rôle
`__server`	`'http://127.0.0.1'`	hôte
`__env_port`	`':3334'`	**vide en production**
`__env`	`__server . __env_port . '/'`	la racine de toute URL construite
`__env_front`	`'http://localhost:5173/'`	l'application front séparée, si elle existe
`__csrf_origin`	liste séparée par des virgules	les origines autorisées en CORS
`__prod`	`false`	bascule le comportement d'erreur (voir §4.4)
`__route_cache`	`false`	active le cache de routes (`cache/routes.php`)
`__default_lang` / `__lang`	`"fr"`	langue par défaut

Chemins

Constante	Valeur
`ROOT`	la racine du projet

Constante	Valeur
`UPLOAD_DIR`	`ROOT . 'uploads/'`
`admin_dir` / `web_dir`	`ROOT . 'admin/'` / `ROOT . 'web/'`
`CLASS_EXTEND`	`ROOT . 'dclass/extends'`
`CLASSJS`	`__env . 'dclass/devupsjs/'`
`assets`	`__env . 'web/assets/'` (déclarée dans `header.php`)
`__admin`	`__env . 'admin/assets/'`

Clés de session

Constante	Valeur	Rôle
`ADMIN`	`__project_id . '_devups'`	la clé de `$_SESSION` qui porte l'administrateur connecté
`CSRFTOKEN`	`__project_id . '_csrf_token'`	
`dv_role_navigation` / `dv_role_permission`	`__project_id . '_navigation'` / `'_permission'`	droits en session

Le préfixe `__project_id` est ce qui permet à deux projets Devups servis depuis le même hôte de ne pas se marcher sur les sessions.

Secrets et services

Famille	Constantes
JWT	`jwt_secret_Key`, `jwt_expire_time` (`false` = jeton sans expiration)
SMTP	`sm_port`, `sm_smtp`, `sm_username`, `sm_password`, `sm_from`, `sm_name`, `sm_smtpsecurity`
Notifications push (FCM)	`fcm_pair_key`, `fcm_user_id`, `fcm_project_id`, `fcm_jwt_auth_file`
Google	`GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_ID_WEB`, `GOOGLE_CLIENT_ID_ANDROID`, `GOOGLE_CLIENT_ID_IOS`, `GOOGLE_CLIENT_SECRET`, `GOOGLE_REDIRECT_URL`
Tâches planifiées	`cron_token`

> **`jwt_expire_time = false`** désactive toute vérification d'expiration : `Auth::execute()` (`Auth.php:113`) ne compare `iss` / `nbf` / `exp` que si la constante est vraie, et `Auth::getJWT()` n'émet alors ni `iat`, ni `nbf`, ni `exp`. Un jeton émis est valable indéfiniment. C'est l'état de 3agReader.

4.3 L'ordre de chargement

`config/dependanceInjection.php` est le seul point commun à tous les points d'entrée. Il fait, dans cet ordre :

1. `require config/constante.php` ;

2. déclare les constantes applicatives qui doivent survivre à un clone ;
3. `require vendor/autoload.php` si `vendor/` existe ;
4. `require config/Database.php` ;
5. `require` **explicitement, un par un**, tous les fichiers de `dclass/` — l'ordre est significatif (`Model-Serializer` avant `ModelTrait` avant `Model`, `QueryBuilder` avant `Controller`) ;
6. `$em = DBAL::getEntityManager()` — l'`EntityManager` Doctrine, global.

Il n'y a **pas d'autochargement paresseux du framework** : tout `dclass/` est en mémoire à chaque requête.

Puis `header.php` (application publique) ajoute :

```
session_start();
require config/dependanceInjection.php;
require lang.php;
require src/requires.php;        // tous les modules du projet
require route/WebService.php;
require route/App.php;

$dvlangs = Dvups_lang::all();    // une requête SQL, à chaque requête HTTP
$viewdir = [web_dir . "views"];
$global_config = require ROOT.'config/dvups_configurations.php';
```

`admin/header.php` fait l'équivalent pour le back-office.

> `src/requires.php` est la liste des modules du projet. **Un nouveau module doit y être ajouté à la main** : sans cela, `Fatal error: Class XController not found`.

4.4 Le mode production

`__prod` change le comportement sur trois points mesurés :

Situation	`__prod = false`	`__prod = true`
route introuvable (page)	`var_dump(" You may create method XView in entity. ")`	`Genesis::render('404')`
route introuvable (API)	le même message, en JSON	`{"success":false,"message":"404 : ... service note found"}`
`dv_dump()`	affiche et **tue la requête** (`die(1)`)	identique — `dv_dump()` tue la requête dans les deux cas

`dv_dump()` (`dvups_functions.php`) est `dump($args); die(1);`. **Ce n'est pas un journal, c'est un arrêt.** Tout code de framework qui veut signaler un problème sans tuer la requête doit utiliser `error_log()` — c'est la règle que les protocoles du Tome IV appliquent.

Chapitre 5 — Le cycle de vie d'une requête

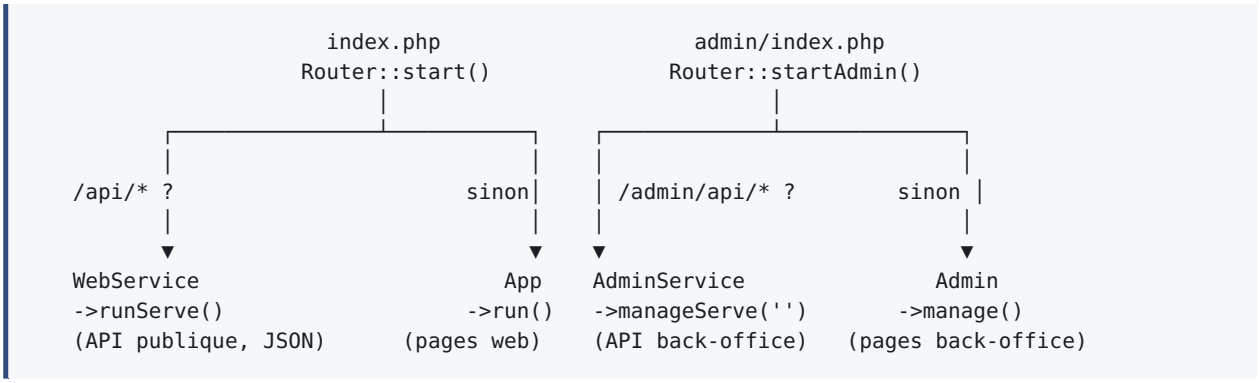
C'est le chapitre à lire avant tous les autres : la plupart des surprises de ce framework viennent de ce que **quatre dispatchers différents** cohabitent, et qu'ils ne suivent pas les mêmes règles.

5.1 Les deux points d'entrée

Fichier	Contenu	Appelle
`index.php`	`require 'header.php'; Router::start();`	l'application publique
`admin/index.php`	équivalent	`Router::startAdmin()`

`Router::getPathInfo()` (`Router.php:66`) établit `self::$path_info` en essayant, dans l'ordre : `$_SERVER['PATH_INFO']`, puis `REQUEST_URI` moins le répertoire de base, puis `$_GET['url']`, puis la chaîne de requête. La partie après `?` est retirée.

5.2 Les quatre portes



Les quatre classes vivent dans `route/` et héritent de `Router` (ou, pour `Admin`, de `ModuleController` qui en hérite).

5.3 Porte 1 — `App::run()`, les pages publiques

`Router::run()` (`Router.php:158`) :

1. **Langue dans l'URL.** Si les deux premiers caractères du chemin correspondent à un code ISO de langue connu (`$dlangs`, peuplé dans `header.php`), ils sont retirés du chemin et `Router::$lang` est positionné. `/fr/comicbook` et `/comicbook` mènent donc au même endroit, dans deux langues.
2. **Cache de routes**, si `__route_cache` est vrai (voir §5.8).
3. `self::$path` = le **premier segment** du chemin, ou `$default_url` (`'hello'`) si le chemin est vide.

4. **Le nom de méthode** : le segment est découpé sur `-` et `_`, chaque morceau passe en capitale initiale, et `"View"` est ajouté.

```
/mon-compte       →  MonCompteView()
/comicbook        →  ComicbookView()
/                 →  HelloView()
```

1. Si la méthode n'existe pas sur `App`, le tableau `$append_routes` est consulté — c'est le seul mécanisme de route nommée du framework :

```
// route/App.php
public $append_routes = [
    '/printRib' => 'Accountrequests@printRib'
];
```

1. Sinon : 404 en production, `var_dump` en développement.
2. `mapFunction('App', $function)` (voir §5.7).

5.4 Porte 2 — `WebService::runServe()`, l'API publique

`Router::runServe()` (`Router.php:346`) fait, dans l'ordre :

a. CORS. L'origine de la requête est comparée à `__csrf_origin`. Si elle y figure (ou vaut `null`), les en-têtes `Access-Control-Allow-Origin` et `-Credentials` sont émis. Les en-têtes autorisés sont codés en dur :

```
Authorization, Auth, X-Customer-Auth, Columns, Dlang, common,
Content-Type, X-Requested-With
```

Une requête `OPTIONS` répond `200` et s'arrête là.

> **Piège de développement.** Si le serveur de développement du front choisit un autre port que ceux listés dans `__csrf_origin`, l'appel échoue en « CORS policy: No 'Access-Control-Allow-Origin' » — côté navigateur le message n'est qu'un « Failed to fetch ». Ajouter le port à `__csrf_origin`.

b. Deux en-têtes lus comme des paramètres.

En-tête	Devient	Usage
`Co-lumns`	`Request::$uri_hea-der_param['dcolumns']`	un JSON `{"relation":["col1","col2"]}` qui déclenche un `with()` sans toucher au code (voir Tome II)
`Dlang`	`Request::$uri_hea-der_param['dlang']`	force la langue de la réponse

c. L'authentification, inconditionnellement. `(new Auth())->execute()` est appelé **avant tout dispatch**. S'il réussit, `Auth::$user_id` est peuplé et `Request::$uri_get_param['session_user_id']` est posé. S'il échoue, rien ne se passe : l'absence de jeton n'est pas une erreur à ce stade.

d. Le dispatch. Le chemin, privé de `/api/`, est découpé sur `/` :

Condition	Dispatch	Style
≥ 2 segments **et** segment 0 ∈ `create, upload, detail, read, update, delete, lazyloading`	`systemServe()`	**générique**
segment 0 est une clé de `$public_access`	`FrontController::frontServe()`	**REST dédié**
sinon	`<Segment>Serve()` sur `WebService`	**service libre**

```
POST /api/create/comicbook       → systemServe()           générique
GET  /api/comicbook/42           → frontServe()            REST dédié
GET  /api/hello                  → helloServe()            service libre
POST /api/accept-invitation?id=5 → acceptInvitationServe()  service libre
```

La conversion du segment en nom de méthode passe par `Util::toPascalCase()` suivi de `"Serve"`.

e. Si la méthode n'existe pas : `$append_routes`, puis 404.

5.5 Porte 3 — `Admin::manage()`, les pages du back-office

`Router::manage()` (`Router.php:448`) positionne `Router::$admin_route = true` — un drapeau qui a des conséquences de sécurité réelles (voir Tome III) — puis :

1. **Session obligatoire.** Si `$_SESSION[ADMIN]` est absent et que le chemin n'est pas `connexion-action`, redirection vers `admin/login.php`.
2. Le chemin est découpé ; `admin` est ajouté en tête s'il manque.
3. **Le nombre de segments décide tout** :

Segments	Interprétation	Conséquence
≥ 5	`/admin/<composant>/<module>/<entité>/<action>`	lit `config/dvups_configurations.php` et appelle `Controller::views($this, $entity)` — **le CRUD générique d'administration**
≥ 3	`/admin/<composant>/<module>`	lit `config/module_configurations.php` et appelle `Controller::viewsModule($this, $module)`
sinon	`/admin/<page>`	méthode `<Page>View()` sur `Admin`

```
/admin/                                    → DashboardView()
/admin/shopping/ModuleOrder                → viewsModule()
/admin/shopping/ModuleOrder/orders/list    → Controller::views()
```

C'est ce mécanisme qui fait qu'une entité générée a un écran de liste, de détail et de formulaire **sans qu'une seule ligne de contrôleur d'écran soit écrite** : le chemin à cinq segments suffit, à condition que l'entité figure dans `dvups_configurations.php`.

5.6 Porte 4 — `AdminService::manageServe()`, l'API du back-office

`Router::manageServe()` (`Router.php:541`) :

Forme	Dispatch
2 segments, segment 0 ∈ les sept verbes génériques	`systemServe()`
2 segments, `<entité>/<action>`	`Controller::serve($this, $entity)`
sinon	`<Path>Service()` sur `AdminService`

Noter la différence de suffixe : **`View` pour une page, `Serve` pour l'API publique, `Service` pour l'API du back-office.**

> `manageServe()` contient un contrôle de session `$_SESSION[ADMIN]` **désactivé** (`Router.php:546-550` : le corps du `if` est commenté). L'API du back-office ne vérifie donc pas la session à ce niveau. Les contrôles subsistent plus bas, dans `Controller::serve()` et dans les droits de rôle — mais l'absence de garde en tête de porte est à connaître.

5.7 `mapFunction()` — le dernier kilomètre

`Router::mapFunction($classname, $function)` (`Router.php:625`) est appelée par trois des quatre portes. Elle :

1. réfléchit la méthode cible (`ReflectionAnnotatedClass`) ;
2. **lie les paramètres** : pour chaque paramètre de la méthode, si `Request::$uri_get_param` contient une clé du même nom, la valeur est passée. Une méthode `detailView($id)` reçoit donc automatiquement `?id=...` ;
3. **contrôle `@Auth`** : si la méthode porte l'annotation, `Auth::$user_id` doit être peuplé, sinon exception ;
4. cherche une annotation **du nom du verbe HTTP courant** (`@GET`, `@POST`, …), sinon `@Api` ; si l'une des deux existe, son `execute()` est appelé pour extraire les paramètres de chemin ;
5. appelle la méthode.

> **Point important, et source de confusion.** Si **aucune** annotation de verbe ni `@Api` n'est trouvée, `mapFunction()` met `$methodView = false` et **continue** : le `405` de l'étape 4 est commenté (`Router.php:666-669`). Une méthode `xxxServe()` / `xxxView()` sans aucune annotation est donc parfaitement appelable.
>
> **Ce n'est pas vrai de l'autre dispatcher.** `Request::FrontController()` (le style REST dédié, §5.4) applique bien le `405` : une méthode de `FrontController` sans annotation de verbe ni `@Api` est **inatteignable**, quel que soit le verbe (`Request.php:577-585`). Les deux dispatchers divergent sur ce point précis, et cette divergence a une conséquence vérifiée sur 3agReader : deux méthodes de `OrdersFrontController` (`confirmOrderWithWallet()` et `subscribeAction()`), publiques, nommées, dépourvues d'annotation, ne s'exécutent jamais — par aucune route.

5.8 Le cache de routes

Si `__route_cache` est vrai, `runCache($url)` (`Router.php:217`) est consulté avant le dispatch normal. Il lit `cache/routes.php`, un tableau indexé par chemin :

```
'/comicbook/42' => [
    'method' => 'ComicbookFrontController@detail',
    'params' => ['id'],
    'route_params' => ['id'],
    'path' => 'detail',
    'dclass' => 'Comicbook',
    'api' => true,
]
```

Les clés spéciales `@admin` et `@auth` enveloppent l'entrée et déclenchent respectivement le chemin d'administration et un contrôle `Auth`.

> **État réel : le cache de routes n'est jamais écrit.** `Router::cacheRoute()` (`Router.php:697`) commence par `return null;` — tout le corps de la fonction est inatteignable. Sur 3agReader, `__route_cache` vaut par ailleurs `false`. Le mécanisme est complet côté lecture, mort côté écriture.

5.9 Résumé en un tableau

Porte	Classe	Point d'entrée	Convention de méthode	405 sur annotation absente
Pages publiques	`App`	`Router::start()`	`<Path>View()`	non
API publique	`WebService`	`Router::start()` + `/api/`	`<Path>Serve()`, ou générique, ou `FrontController`	**oui, pour le seul `FrontController`**
Pages back-office	`Admin`	`Router::startAdmin()`	`<Path>View()`, ou CRUD générique par nombre de segments	non
API back-office	`AdminService`	`Router::startAdmin()` + `/api/`	`<Path>Service()`, ou générique	non

Chapitre 6 — La ligne de commande

6.1 Le binaire

```
// backend/devups
<?php
include ('dclass/generator/builder.php');
```

Tout se passe dans `dclass/generator/builder.php`, un script procédural qui analyse `$argv`.

```
php devups <commande> [cible] [options]
```

Sans argument reconnu, `__Generator::help()` est affichée.

6.2 Les commandes de cycle de vie

Commande	Effet
`install`	crée la base, le schéma, et tente de semer l'administrateur maître (voir §3.2)
`database:create`	crée la base de données seule
`init:caches`	crée `cache/` et `database/` avec leurs sous-dossiers
`serve`	lance le serveur PHP de développement
`tests`	lance les tests
`build`	copie `src/` et `dclass/` dans `build/` et produit `build/<__project_id>.zip`
`deploy`	décompresse ce zip à la racine

6.3 Les commandes de schéma

Commande	Effet
`schema:update --dump-sql`	affiche le SQL que Doctrine appliquerait, sans l'appliquer
`schema:update --force`	l'applique, puis synchronise les entités
`transaction:rollback <id>`	rejoue un dump de `database/dump/dump_<id>.sql` et annule la transaction `<id>`

`schema:update` délègue littéralement à `bin/doctrine orm:schema:update`. La liste des dossiers que Doctrine scanne est **reconstruite à chaque appel** par `DBAL::getEntityManager()`, qui appelle `Core::buildOriginCore()` — voir le chapitre 7.

> **Piège vécu, revécu deux fois (BGFIBank Queue).** `--force` peut répondre `[OK] Database schema updated successfully!` **sans rien appliquer**, sur un renommage d'index provoqué par l'ajout d'une relation à une table déjà en base. `--dump-sql` propose alors la même séquence en boucle, indéfiniment. `SHOW CREATE TABLE` confirme que l'index garde le nom que MySQL lui a donné (`FK_...`) au lieu de celui que Doctrine attend (`IDX_...`). **Contournement** : exécuter le SQL proposé par `--dump-sql` directement en base, pas via `--force`. Le piège touche **tout ajout de relation à une table déjà existante**, pas une entité en particulier.

6.4 Les commandes du datacore

Commande	Effet
`dvups_:update`	**la commande la plus courante** : rescanne `src/`, réécrit `config/dvups_configurations.php` et `config/module_configurations.php`, et met à jour les composants/modules/entités du rôle 1
`dvups_:admin`	rejoue `config_data.sql` (voir §3.2)
`dvups_:implement`	
`entity:g:core <cible> [--sync]`	(re)génère le `<Entité>Core.json` d'une entité existante
`entity:e:data <cible>`	
`entity:g:postmandoc <cible>`	génère une documentation Postman

`php devups dvups_:update` est à lancer après **toute** création ou suppression de module ou d'entité. Son message de retour a longtemps menti (« already uptodate » quoi qu'il arrive) ; sur 3agReader, `Core::updateDvupsTable()` compare désormais le contenu généré à ce qui est sur le disque avant d'écrire, et le message est fiable.

6.5 Les commandes de génération

La cible s'écrit `<Composant>\<Module>\<entité>` (antislashs) :

```
php devups core:g:crud edition3ag\ModuleComicbook\comicbook
```

Commande	Génère
`core:g:component <composant>`	**tout** un composant, depuis son `<composant>Core.json`
`core:g:module <module>`	la coquille d'un module
`core:g:crud <cible>`	entité + formulaire + liste + contrôleur d'administration
`core:g:entity <cible>`	la classe `Entity/<Entité>.php` seule
`core:g:entity <cible> --lang`	la classe `Entity/<Entité>_lang.php`

Commande	Génère
`core:g:form <cible>`	`Form/<Entité>Form.php`
`core:g:table <cible>`	`Datatable/<Entité>Table.php`
`core:g:controller <cible>`	`Controller/<Entité>Controller.php` (back-office)
`core:g:frontcontroller <cible>`	`Controller/<Entité>FrontController.php` (API REST)
`core:g:views <cible>`	les vues Blade
`core:g:formwidget <cible>`	un formulaire en widget réutilisable
`core:g:viewswidget <cible>`	une vue de détail en widget
`core:g:genesis <cible>`	
`core:g:dependencies <cible>`	les `require` d'une entité
`core:g:moduleendless`, `core:g:moduledependencies`, `core:g:moduleindex`, `core:g:moduleservices`, `core:g:moduleressources`	les pièces d'un module, séparément

`core:g:crud` ne génère jamais de `FrontController`. Il faut `core:g:frontcontroller` pour cela, et ensuite décommenter sa ligne `require` dans `<composant>.module<nom>.php` (elle est générée commentée).

6.6 Le garde de régénération, et l'option `--force`

Un générateur qui écrase un fichier écrit à la main détruit du travail. `GeneratorFileGuard` (`dclass/generator/GeneratorFileGuard.php`, ajouté sur 3agReader) intercepte cela : **avant d'écrire un fichier qui existe déjà, il le renomme** en une version horodatée, et ne génère que si le renommage a réussi.

```
private static function renameIfPresent($path)
{
    if (!file_exists($path)) return true;
    if (self::$force) return true;
    $old = self::uniquePath($path);
    if (!@rename($path, $old)) {
        self::reportFailure($path, "rename() a echoue vers $old "
            . "(fichier probablement ouvert ailleurs)");
        return false;
    }
    self::$renamed[] = ['new' => $path, 'old' => $old];
    return true;
}
```

`--force` désactive le renommage et écrase. L'option est cherchée **n'importe où dans `$argv`**, pas à une position fixe — parce que `core:g:entity` utilise déjà `$argv[3]` pour `--lang` et `core:g:crud` pour le paquet Java.

Après les treize commandes `core:g:*`, `GeneratorFileGuard::printSummary()` affiche ce qui a été renommé et ce qui a échoué.

6.7 Ce qu'il faut savoir avant de régénérer

Ces quatre faits ont chacun coûté une session de débogage sur un projet réel.

1. Régénérer une entité existante duplique ses `require`. `core:g:crud` sur une entité déjà générée **ajoute un second bloc de `require`** à la fin de `<composant>.module<nom>.php` au lieu de laisser le premier. Comme ce sont des `require` et non des `require_once`, le résultat est un `Fatal error: Cannot redeclare class X` **potentiel** — il dépend de l'ordre de chargement, donc il n'apparaît pas toujours. **Toujours relire le fichier de module après un `core:g:crud` sur une entité existante.**

2. Toute entité éditée à la main pour une contrainte sortira de la régénération. Une `@UniqueConstraint`, un `nullable=false` sur un `@JoinColumn`, une identité composite : tout cela est écrasé par `core:g:crud`, qui revient à la forme générée. L'entité passe alors en **édition manuelle permanente**, et cela se compte : sur BGFIBank Queue, 14 entités étaient dans ce régime.

3. Le `nullable` d'une relation est silencieusement ignoré par le générateur. Dans `BackendGenerator.php`, les deux points où `@JoinColumn` est produit interpolent `$relation->ondelete` mais **jamais `$relation->nullable`**. Que le datacore déclare `"nullable":"not"` ou `"nullable":"default"` sur une relation `manyToOne`, l'annotation générée est toujours `@JoinColumn(onDelete="cascade")`, et Doctrine applique son défaut, `nullable=true`. **La colonne de clé étrangère est physiquement NULLable dans tous les cas**, quoi que le datacore prétende. Une relation réellement obligatoire demande d'ajouter `nullable=false` à la main — donc de passer l'entité en édition manuelle (point 2).

4. Le `update()` généré n'écrit rien. Le gabarit produit :

```
$x = $this->hydrateWithJson(new Agency_service($id), $rawdata["agency_service"]);
```

`Model::hydrate()` pose `$this->dvold = $this` — **une référence au même objet**, pas un clone. `entityKeyUpdate()` décide quels champs écrire en comparant `$val != $this->dvold->{$key}` : comme `dvold` **est** `$this`, modifier un champ modifie aussi sa propre valeur « avant », au même instant. La comparaison ne détecte donc jamais de différence, et la réponse est `{"success":false,"detail":"Empty parameters"}` avec un `200 OK`. Un `print_r()` sur une entité hydratée affiche `[dvold] => NomEntite Object *RECURSION*` : PHP détecte lui-même la boucle.

`Entity::find($id)` n'a pas ce problème : il passe par `DBAL::__findOneRow()`, qui pose `$flowBD->dvold = clone $flowBD`, un vrai clone.

Le correctif retenu (arbitrage de l'auteur du framework) est dans le code généré, pas dans le framework :

```
$x = $this->hydrateWithJson(Agency_service::find($id), $rawdata["agency_service"]);
```

`Model::hydrate()` n'est **pas** corrigé (`$this->dvold = clone $this` marcherait aussi, c'est testé) parce que cela touche le cœur du framework avec un risque de régression sur tout projet basé dessus. **Conséquence : le correctif est à réappliquer après chaque `core:g:crud`**, tant que le gabarit lui-même n'est pas corrigé.

Chapitre 7 — Le générateur et le scan du disque

7.1 Les trois niveaux de datacore

Niveau	Fichier	Emplacement
Composant	`<composant>Core.json`	racine du composant
Module	`<module en minuscules>Core.json`	racine du module
Entité	`<Entité>Core.json`	dans `Core/` du module

Le Tome II décrit le **contenu** de ces fichiers. Ce chapitre décrit comment ils sont **lus**.

7.2 Le scan, mesuré

`Core::buildOriginCore()` (`Core.php:147`) parcourt `src/` et, pour chaque dossier, appelle `Core::findprojectcore()` :

`findprojectcore($dir, $file)` (`Core.php:63`) 1. Si `$dir/<file en minuscules>Core.json` n'existe pas → **retourne `[]`**. Le composant n'existe pas pour le framework. 2. Lit ce fichier. 3. **Scanne les sous-dossiers** et appelle `findmodulecore()` sur chacun. 4. **Écrase `$projectcore->listmodule`** par le résultat du scan.

`findmodulecore($dir, $file)` (`Core.php:96`) 1. Si `$dir/<file en minuscules>Core.json` n'existe pas → **retourne `[]`**. 2. Lit ce fichier. 3. Appelle `findentitycore($dir . "/Core")` et pose le résultat dans `listentity`.

`findentitycore($dir)` (`Core.php:114`) - Pour chaque fichier de `Core/` : les dossiers sont ignorés, le reste est décodé en JSON.

7.3 Les trois conséquences à retenir

1. `listmodule` dans un `<composant>Core.json` est ignoré. Il est écrasé par le scan des sous-dossiers (`Core.php:91`). **Éditer ce tableau à la main ne sert à rien.**

2. Un module sans son `*Core.json` est invisible, sans erreur. Pour qu'un nouveau module et ses entités soient repérés par `schema:update`, il faut **et il suffit** de créer `src/<composant>/<Module>/<module en minuscules>Core.json` avec au minimum `{"name":"ModuleXxx"}`. Sans ce fichier, `schema:update --dump-sql` ne montre **aucun diff** pour les nouvelles tables — silencieusement.

3. Les `<Entité>Core.json` individuels ne pilotent pas le scan Doctrine. Une fois le module trouvé, **tout** fichier `.php` annoté `@Entity` dans son dossier `Entity/` est pris en compte par Doctrine, qu'il ait un `Core/<Entité>Core.json` ou non. Ces fichiers servent à la génération de code et à une collection annexe, pas au scan.

7.4 Créer un module à la main

La séquence minimale, mesurée :

1. `src/<composant>/<Module>/<module>Core.json` → le module existe.
2. Les entités dans `Entity/`, annotées `@Entity`.
3. `php devups schema:update --dump-sql` puis `--force` → les tables.
4. `require 'component/ModuleXxx/moduleXxx.php';` dans `src/requires.php` → les classes sont chargées.
5. `php devups dvups_:update` → l'entité apparaît dans le back-office.
6. Pour l'API REST dédiée : enregistrer l'entité dans `$public_access` de `route/WebService.php` (voir Tome III).

Les étapes 4 et 6 sont indépendantes : l'une sert au chargement des classes, l'autre à l'exposition de la route.

7.5 Ce que `dvups_:update` écrit vraiment

`Core::updateDvupsTable()` (`Core.php:184`) :

- construit les deux tableaux de configuration décrits en §2.5 ;
- les écrit via `Util::arrayToPhpFile()` ;
- puis **modifie le rôle 1** : `setComponents()`, `setModules()`, `setEntities()`, `updateConfigs()`, `__update()`.

Le rôle 1 est l'administrateur maître. C'est par lui que la barre latérale du back-office sait quels modules afficher. Un module nouvellement créé n'apparaît donc dans le menu **qu'après** `dvups_:update`.

`Util::arrayToPhpFile()` (`Util.php:228`) écrit de façon **atomique et assainie** — ce point a été durci sur 3agReader :

```
if (!preg_match('/^[A-Za-z0-9_\-]+(\.[A-Za-z0-9_\-]+)*$/', $filename)) {
    throw new \InvalidArgumentException(...);
}
$targetPath = ROOT . $folder . "$filename.php";
$tmpPath = $targetPath . '.' . uniqid('tmp_', true);
file_put_contents($tmpPath, $phpContent);
if (!rename($tmpPath, $targetPath)) {
    @unlink($tmpPath);
    throw new \RuntimeException(...);
}
```

La regex autorise les séparateurs de la forme `<nom>.<iso>` mais interdit `../` et tout caractère hors de la liste. Le fichier temporaire suivi de `rename()` garantit qu'aucun lecteur ne voit un fichier à moitié écrit. Cette fonction est aussi celle qui écrit le cache d'arbres du Tome IV — d'où le soin.

7.6 Les trois choses que le datacore ne sait pas exprimer

Vérifié en lisant l'intégralité de `BackendGenerator.php` :

Besoin	État	Solution
contrainte d'unicité	**aucune notion dans le format JSON** — le seul mot « unique » du générateur est un commentaire sans rapport	`@UniqueConstraint` ou `unique=true` à la main dans l'annotation de l'entité
relation obligatoire	`"nullable":"not"` est **ignoré** sur une relation (voir §6.7)	`nullable=false` à la main dans `@JoinColumn`
identité composite	aucune notion	`@Id @ManyToOne` à la main, pas de colonne `id`

Dans les trois cas, l'entité passe en édition manuelle permanente.

> **Sur la contrainte d'unicité, une leçon transférable.** Poser une contrainte unique **hors** du pipeline (un script rejoué après chaque `--force`) fonctionne mais produit un `DROP INDEX` **permanent** dans `--dump-sql` : Doctrine ne connaît pas cet index et propose de le supprimer, à chaque appel, pour toujours. Le critère de recette « aucun diff en attente » se dégrade alors en « aucun diff au-delà de N lignes connues », et un vrai diff devient facile à manquer.
>
> La bonne forme nomme la contrainte **à l'identique de l'index physique déjà en base** :
>
> `php /** * @Entity @Table(name="agency", uniqueConstraints={ * @UniqueConstraint(name="uniq_agency_code", columns={"code"}) * }) */`
>
> Doctrine reconnaît alors l'index existant comme sien : **aucune migration n'est nécessaire**, et `--dump-sql` cesse simplement de proposer le `DROP INDEX`. À l'inverse, `@Column(..., unique=true)` **sans nom explicite** fait générer par Doctrine son propre nom (`UNIQ_...`), différent de l'existant — ce qui provoque un vrai `DROP INDEX` suivi d'un `CREATE UNIQUE INDEX`, évitable.
>
> **Piège de docblock, vécu.** `AnnotationDriver` ne lit que le docblock **immédiatement adjacent** à `class Xxx`. Intercaler un docblock de prose (`/** ... */`) entre l'annotation et la classe fait échouer **toute** lecture de métadonnées (« Class ... is not a valid entity or mapped super class »). L'explication va dans un commentaire ordinaire `/* ... */`, séparé, juste avant.

Chapitre 8 — Premier module, de bout en bout

Ce chapitre assemble les sept précédents sur un exemple complet : un module `ModuleTask` dans un composant `monprojet`, avec une entité `task` liée à un `project`.

8.1 Le datacore du composant

`src/monprojet/monprojetCore.json` :

```
{
  "name": "monprojet",
  "template": "adminv2",
  "listmodule": []
}
```

`listmodule` peut rester vide : il est écrasé par le scan (§7.3).

8.2 Le datacore du module

`src/monprojet/ModuleTask/moduletaskCore.json` :

```
{
  "name": "ModuleTask",
  "label": "Tâches",
  "icon": "fas fa-fw fa-check-square"
}
```

`label` et `icon` sont lus par `Core::updateDvupsTable()` (`Core.php:264-265`) et alimentent la barre latérale du back-office. Sans eux, le nom du module et une icône d'engrenage par défaut sont utilisés.

8.3 Le datacore de l'entité

`src/monprojet/ModuleTask/Core/taskCore.json` :

```
{
  "name": "task",
  "label": "Tâche",
  "attribut": {
    "1": { "name": "name",        "datatype": "string",  "size": "255", "nullable": "not",
 "formtype": "text" },
    "2": { "name": "description", "datatype": "text",                  "nullable":
"default", "formtype": "textarea" },
    "3": { "name": "done",        "datatype": "integer",               "nullable":
"default", "formtype": "checkbox" }
  },
  "relation": [
    { "entity": "project", "cardinality": "manyToOne", "nullable": "default", "ondelete": "cas
cade" }
  ]
}
```

Deux détails qui comptent :

- `attribut` est un **objet indexé** (`"1"`, `"2"`, …), pas un tableau ;
- `id` est implicite. **Ne jamais le déclarer dans `attribut`.**

8.4 Générer

```
php devups core:g:crud monprojet\ModuleTask\task
php devups core:g:frontcontroller monprojet\ModuleTask\task
```

Ce qui apparaît :

```
Entity/Task.php                    l'entité, avec ses annotations Doctrine
Form/TaskForm.php                  le formulaire déclaratif
Datatable/TaskTable.php            la liste déclarative
Controller/TaskController.php      le CRUD de back-office
Controller/TaskFrontController.php l'API REST dédiée
monprojet.moduletask.php           les require
```

8.5 Les quatre gestes manuels

Ils ne sont pas optionnels.

1. Décommenter le `require` du `FrontController` dans `monprojet.moduletask.php` — il est généré commenté.

2. Ajouter le module à `src/requires.php` :

```
require 'monprojet/ModuleTask/monprojet.moduletask.php';
```

3. Corriger le `update()` généré (§6.7, point 4) :

```
// remplacer
$task = $this->hydrateWithJson(new Task($id), $rawdata["task"]);
// par
$task = $this->hydrateWithJson(Task::find($id), $rawdata["task"]);
```

4. Enregistrer l'entité pour l'API, dans `route/WebService.php` :

```
public $public_access = [
    // ...
    'task' => \Task::class,
];
```

Sans cet enregistrement, `GET /api/task` répond comme une entité inconnue.

8.6 Migrer la base

```
php devups schema:update --dump-sql    # lire avant d'appliquer
php devups schema:update --force
php devups dvups_:update               # pour que l'écran apparaisse au back-office
```

Toujours lire `--dump-sql` avant `--force`. Et si `--dump-sql` propose la même chose après un `--force` qui a répondu « success », appliquer le SQL directement en base (§6.3).

8.7 Vérifier

Chemin	Attendu
`/admin/monprojet/ModuleTask/task/list`	la liste, avec les trois colonnes
`GET /api/task`	la liste paginée, `{"success":true,...}`
`GET /api/task/1`	le détail
`POST /api/task` avec `{"task":{"name":"x","project.id":3}}`	la création
`php devups schema:update --dump-sql`	`Nothing to update`

Noter la forme `"project.id": 3` dans la charge utile : le point référence une relation `manyToOne`. Le Tome II y revient.

8.8 Si quelque chose ne marche pas

Symptôme	Cause la plus probable
`schema:update --dump-sql` ne voit pas la nouvelle table	le `<module>Core.json` manque (§7.3)
`Fatal error: Class TaskController not found`	`src/requires.php` n'a pas été mis à jour
`Fatal error: Cannot redeclare class Task`	le double bloc de `require` d'une régénération (§6.7, point 1)

Symptôme	Cause la plus probable
`Class 'Task' does not exist` alors que le fichier est là	un classmap Composer hérité d'un autre projet — `composer dump-autoload` (§1.4)
l'entité n'apparaît pas au back-office	`php devups dvups_:update` n'a pas été lancé
`GET /api/task` répond comme une entité inconnue	`$public_access` (§8.5, geste 4)
une mise à jour répond `200` avec `success:false, "Empty parameters"`	le `new X($id)` du gabarit (§6.7, point 4)
la colonne de clé étrangère accepte `NULL` malgré `"nullable":"not"`	le générateur ignore `nullable` sur une relation (§6.7, point 3)

Fin du Tome I. Le Tome II traite des données : le datacore en détail, les entités, le `QueryBuilder`, l'écriture, les transactions et la traduction.